

AI Application Lecture 9

Neural Network Compression and Distance Measures between Probabilistic Language Models

SUZUKI, Atsushi^a

^aI thank my collaborator, Jing WANG, for cooperating in creating the teaching materials.

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How can we compress a NN? - Low precision and quantization
4. How can we evaluate the model modification? - Divergence
5. Summary

Review of the previous lecture

In the previous lecture, we rigorously defined and calculated frameworks for the **automatic and quantitative evaluation** of a **single probabilistic language model**.

Review of the previous lecture

In the previous lecture, we rigorously defined and calculated frameworks for the **automatic and quantitative evaluation** of a **single probabilistic language model**.

We covered:

- Perplexity
- Accuracy of the most likely option in multiple-choice questions
- Various metrics for string output (EM/F1, BLEU, ROUGE, chrF, BERTScore, numerical Accuracy)

Learning outcomes for this lecture

By the end of this lecture, you should be able to:

- Explain the motivation and methods to **reduce the scale (model compression)** of a **neural network** while maintaining its properties as a **function**.

Learning outcomes for this lecture

By the end of this lecture, you should be able to:

- Explain the motivation and methods to **reduce the scale (model compression)** of a **neural network** while maintaining its properties as a **function**.
- When a **probabilistic language model** is modified, evaluate the **amount of change from the original model** in a **mathematically rigorous** and **quantitative** manner.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. How can we compress a NN? - Low precision and quantization

4. How can we evaluate the model modification? - Divergence

5. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. How can we compress a NN? - Low precision and quantization

4. How can we evaluate the model modification? - Divergence

5. Summary

Preliminaries: mathematical notations

Set:

- Sets: $\mathcal{A}$
- Membership: $x \in \mathcal{A}$
- Empty set: $\{\}$
- Roster notation: $\{a, b, c\}$
- Set-builder: $\{x \in \mathcal{A} | P(x)\}$
- Cardinality: $|\mathcal{A}|$
- Real numbers: $\mathbb{R}, \mathbb{R}_{>0}, \mathbb{R}_{\geq 0}$
- Integers: $\mathbb{Z}, \mathbb{Z}_{>0}, \mathbb{Z}_{\geq 0}$
- Integer range: $[1, k]_{\mathbb{Z}} := \{1, \dots, k\}$

Function:

- $f : \mathcal{X} \rightarrow \mathcal{Y}$: f maps from $\mathcal{X}$ to $\mathcal{Y}$.
- $y = f(x)$: The output of f for input x .

Definition:

- (LHS) $:=$ (RHS): Left side is defined by the right side.

Preliminaries: mathematical notations

Sequence: Denoted by $\mathbf{a} = (a_1, a_2, \dots)$.

- A function $\mathbf{a} : [1, n]_{\mathbb{Z}} \rightarrow \mathcal{A}$.
- Length is denoted by $|\mathbf{a}|$.

Vector: Denoted by $\mathbf{v}$.

- A column of numbers, $\mathbf{v} \in \mathbb{R}^n$.
- i -th element is v_i .

Matrix: Denoted by $\mathbf{A}$.

- $m \times n$ matrix: $\mathbf{A} \in \mathbb{R}^{m,n}$.
- (i, j) -th element is $a_{i,j}$.
- Transpose: $\mathbf{A}^{\top}$.

Tensor: Denoted by $\underline{\mathbf{A}}$.

- Simply a multi-dimensional array.
- Vector $\rightarrow$ 1st-order, Matrix $\rightarrow$ 2nd-order.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. How can we compress a NN? - Low precision and quantization

4. How can we evaluate the model modification? - Divergence

5. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. How can we compress a NN? - Low precision and quantization

4. How can we evaluate the model modification? - Divergence

5. Summary

Question: Practical NN-based models consume memory and storage heavily. It largely comes from parameter representations.

How

Question: Practical NN-based models consume memory and storage heavily. It largely comes from parameter representations.

How can we reduce AI parameter memory/storage consumption, without compromising performance?

Revisiting the formulation of AI as function learning

The primary goal of AI was to **learn an appropriate relationship (function) between inputs and outputs.**

Revisiting the formulation of AI as function learning

The primary goal of AI was to **learn an appropriate relationship (function) between inputs and outputs.**

However, even after a good function has been obtained, there is a strong practical motivation to modify it to reduce **implementation resources** (memory, computational complexity, latency) while **preserving the input-output relationship of that function as much as possible.**

Revisiting the formulation of AI as function learning

The primary goal of AI was to **learn an appropriate relationship (function) between inputs and outputs.**

However, even after a good function has been obtained, there is a strong practical motivation to modify it to reduce **implementation resources** (memory, computational complexity, latency) while **preserving the input-output relationship of that function as much as possible.**

This process is what we call model **compression.**

Motivation and overview of compression

Even when a good function f has already been obtained, there is motivation to make the **model's representation (parameterization)** more lightweight while preserving the **function values (input-output relationship)**.

Motivation and overview of compression

Even when a good function f has already been obtained, there is motivation to make the **model's representation (parameterization)** more lightweight while preserving the **function values (input-output relationship)**.

Representative methods include:

- **Low-precision floating point**
- **Quantization**
- **Model distillation** [9, 6, 5, 4, 2]

Compression using low-precision floating point

Definition (Low-precision floating number system)

Let $d_{\text{param}} \in \mathbb{Z}_{>0}$. For each index $i \in \{1, \dots, d_{\text{param}}\}$, fix a finite set $\mathbb{F}_i \subset \mathbb{R}$ as a low-precision **floating-point format** and a **rounding operator** $R_i : \mathbb{R} \rightarrow \mathbb{F}_i$.

Compression using low-precision floating point

Definition (Low-precision floating number system)

Let $d_{\text{param}} \in \mathbb{Z}_{>0}$. For each index $i \in \{1, \dots, d_{\text{param}}\}$, fix a finite set $\mathbb{F}_i \subset \mathbb{R}$ as a low-precision **floating-point format** and a **rounding operator** $R_i : \mathbb{R} \rightarrow \mathbb{F}_i$.

The **low-precision mapping** $\Phi_{\text{fp}} : \mathbb{R}^{d_{\text{param}}} \rightarrow \mathbb{F}_1 \times \mathbb{F}_2 \times \dots \times \mathbb{F}_{d_{\text{param}}}$ is defined as

$$\Phi_{\text{fp}}(\theta) := (R_1(\theta_1), R_2(\theta_2), \dots, R_{d_{\text{param}}}(\theta_{d_{\text{param}}})) \quad (1)$$

The method of replacing θ with $\Phi_{\text{fp}}(\theta)$ is called **low-precision conversion** (i.e., using $f_{\Phi_{\text{fp}}(\theta)}$ instead of f_{θ} in the inference phase).

Compression using low-precision floating point

Remark

Equations allow for a mixed type where the format differs for each **element** (**per-parameter**).

Compression using low-precision floating point

Remark

Equations allow for a mixed type where the format differs for each **element (per-parameter)**.

In **mixed-precision training**, different formats $\mathbb{F}_i$ are assigned to gradients, gradient accumulations, weights, and activations to reduce computational resources [9].

Compression using quantization

Definition (General Form of Integer Quantization)

Given a parameter vector θ , we partition its indices. For each partition, we define a **quantization mapping** Q_j and a **dequantization mapping** $\tilde{Q}_j$.

Compression using quantization

Definition (General Form of Integer Quantization)

Given a parameter vector θ , we partition its indices. For each partition, we define a **quantization mapping** Q_j and a **dequantization mapping** $\tilde{Q}_j$.

$$Q_j(x) := \text{clip}_{[-M_j, M_j]} \left(\text{round} \left(\frac{x}{s_j} \right) + b_j \right), \quad (2)$$

$$\tilde{Q}_j(n) := s_j (n - b_j) \quad (n \in \mathbb{Z}) \quad (3)$$

where s_j is a scale, b_j is a zero-point, and M_j is a clipping value.

Compression using quantization

Definition (General Form of Integer Quantization)

Given a parameter vector θ , we partition its indices. For each partition, we define a **quantization mapping** Q_j and a **dequantization mapping** $\tilde{Q}_j$.

$$Q_j(x) := \text{clip}_{[-M_j, M_j]} \left(\text{round} \left(\frac{x}{s_j} \right) + b_j \right), \quad (2)$$

$$\tilde{Q}_j(n) := s_j (n - b_j) \quad (n \in \mathbb{Z}) \quad (3)$$

where s_j is a scale, b_j is a zero-point, and M_j is a clipping value.

The quantized parameter vector is

$\text{Quantize}(\theta) = (Q_{j(1)}(\theta_1), Q_{j(2)}(\theta_2), \dots, Q_{j(d_{\text{param}})}(\theta_{d_{\text{param}}}))$, where $j(i)$ indicates the partition index that the parameter θ_i belongs to.

The quantized model is $f_{\text{Quantize}(\theta)}$.

Compression using quantization

Remark

Determining quantization parameters (s, b) based on data statistics after training is called **post-training quantization** (PTQ). Optimizing them during the training process is called **quantization-aware training** (QAT) [6, 5].

Compression using quantization

Remark

Determining quantization parameters (s, b) based on data statistics after training is called **post-training quantization** (PTQ). Optimizing them during the training process is called **quantization-aware training** (QAT) [6, 5].

Remark

In practice, not just parameters but also **intermediate outputs (activations)** are often quantized. This allows arithmetic to be performed using faster, more efficient integer operations [6].

Compression using quantization

Example (Manual Calculation Example of Quantize and Dequantize)

Given: $d_{\text{param}} = 4$, $\theta' = (1.20, -0.55, 0.07, 2.31)$, and a partition $(\mathcal{I}_1, \mathcal{I}_2) = (\{1, 2\}, \{3, 4\})$.

Parameters for partition 1: $M_1 = 127$, $s_1 = 0.01$, $b_1 = 0$.

Parameters for partition 2: $M_2 = 7$, $s_2 = 0.1$, $b_2 = 1$.

Let's find the quantized vector q and the dequantized vector $\tilde{\theta}$.

Compression using quantization

Component 1 ($i = 1 \in \mathcal{I}_1$):

$$q_1 = \text{clip}_{[-127, 127]}(\text{round}(1.20/0.01) + 0) = \text{clip}(120) = 120$$

$$\tilde{\theta}_1 = 0.01 \cdot (120 - 0) = 1.20$$

Compression using quantization

Component 1 ($i = 1 \in \mathcal{I}_1$):

$$q_1 = \text{clip}_{[-127,127]}(\text{round}(1.20/0.01) + 0) = \text{clip}(120) = 120$$

$$\tilde{\theta}_1 = 0.01 \cdot (120 - 0) = 1.20$$

Component 2 ($i = 2 \in \mathcal{I}_1$):

$$q_2 = \text{clip}_{[-127,127]}(\text{round}(-0.55/0.01)) = \text{clip}(-55) = -55$$

$$\tilde{\theta}_2 = 0.01 \cdot (-55 - 0) = -0.55$$

Compression using quantization

Component 3 ($i = 3 \in \mathcal{I}_2$):

$$q_3 = \text{clip}_{[-7,7]}(\text{round}(0.07/0.1) + 1) = \text{clip}(1 + 1) = 2$$

$$\tilde{\theta}_3 = 0.1 \cdot (2 - 1) = 0.1$$

Compression using quantization

Component 3 ($i = 3 \in \mathcal{I}_2$):

$$q_3 = \text{clip}_{[-7,7]}(\text{round}(0.07/0.1) + 1) = \text{clip}(1 + 1) = 2$$

$$\tilde{\theta}_3 = 0.1 \cdot (2 - 1) = 0.1$$

Component 4 ($i = 4 \in \mathcal{I}_2$):

$$q_4 = \text{clip}_{[-7,7]}(\text{round}(2.31/0.1) + 1) = \text{clip}(23 + 1) = \text{clip}(24) = 7$$

$$\tilde{\theta}_4 = 0.1 \cdot (7 - 1) = 0.6$$

Compression using quantization

From the calculations, the final vectors are:

Quantized vector:

$$\mathbf{q} = (120, -55, 2, 7)$$

Dequantized vector:

$$\tilde{\boldsymbol{\theta}} = (1.20, -0.55, 0.1, 0.6)$$

It can be seen that the large value 2.31 was saturated by the clipping value M_2 , increasing the error.

Compression using quantization

Exercise (Quantize/Dequantize Exercise)

Let $d_{\text{param}} = 3$, $\theta' = (0.34, -1.26, 0.51)$. Partition: $(\mathcal{I}_1, \mathcal{I}_2) = (\{1, 3\}, \{2\})$.

- For $\mathcal{I}_1$: $M_1 = 15$, $s_1 = 0.02$, $b_1 = 2$.
- For $\mathcal{I}_2$: $M_2 = 127$, $s_2 = 0.01$, $b_2 = 0$.

Find q and $\tilde{\theta}$.

Compression using quantization

Answer

Component 1 ($i = 1 \in \mathcal{I}_1$): $q_1 = \text{clip}_{[-15,15]}(\text{round}(0.34/0.02) + 2) = 15$.

$$\tilde{\theta}_1 = 0.02 \cdot (15 - 2) = 0.26.$$

Compression using quantization

Answer

Component 1 ($i = 1 \in \mathcal{I}_1$): $q_1 = \text{clip}_{[-15,15]}(\text{round}(0.34/0.02) + 2) = 15$.

$$\tilde{\theta}_1 = 0.02 \cdot (15 - 2) = 0.26.$$

Component 2 ($i = 2 \in \mathcal{I}_2$): $q_2 = \text{clip}_{[-127,127]}(\text{round}(-1.26/0.01) + 0) = -126$.

$$\tilde{\theta}_2 = 0.01 \cdot (-126 - 0) = -1.26.$$

Compression using quantization

Answer

Component 1 ($i = 1 \in \mathcal{I}_1$): $q_1 = \text{clip}_{[-15,15]}(\text{round}(0.34/0.02) + 2) = 15$.

$$\tilde{\theta}_1 = 0.02 \cdot (15 - 2) = 0.26.$$

Component 2 ($i = 2 \in \mathcal{I}_2$): $q_2 = \text{clip}_{[-127,127]}(\text{round}(-1.26/0.01) + 0) = -126$.

$$\tilde{\theta}_2 = 0.01 \cdot (-126 - 0) = -1.26.$$

Component 3 ($i = 3 \in \mathcal{I}_1$): $q_3 = \text{clip}_{[-15,15]}(\text{round}(0.51/0.02) + 2) = 15$.

$$\tilde{\theta}_3 = 0.02 \cdot (15 - 2) = 0.26.$$

Compression using quantization

Answer

Component 1 ($i = 1 \in \mathcal{I}_1$): $q_1 = \text{clip}_{[-15,15]}(\text{round}(0.34/0.02) + 2) = 15$.

$$\tilde{\theta}_1 = 0.02 \cdot (15 - 2) = 0.26.$$

Component 2 ($i = 2 \in \mathcal{I}_2$): $q_2 = \text{clip}_{[-127,127]}(\text{round}(-1.26/0.01) + 0) = -126$.

$$\tilde{\theta}_2 = 0.01 \cdot (-126 - 0) = -1.26.$$

Component 3 ($i = 3 \in \mathcal{I}_1$): $q_3 = \text{clip}_{[-15,15]}(\text{round}(0.51/0.02) + 2) = 15$.

$$\tilde{\theta}_3 = 0.02 \cdot (15 - 2) = 0.26.$$

Final vectors:

$$\mathbf{q} = (15, -126, 15) \quad \text{and} \quad \tilde{\boldsymbol{\theta}} = (0.26, -1.26, 0.26)$$

The error is large because the 1st and 3rd components were saturated by $M_1 = 15$.

K-quant (double quantization) in GGUF

When searching for GGUF files in Hugging Face, we often find a GGUF file named like “Q4_K_M”.

K-quant (double quantization) in GGUF

When searching for GGUF files in Hugging Face, we often find a GGUF file named like “Q4_K_M”.

- 4 indicates it applied 4-bit quantization for most parameters.

K-quant (double quantization) in GGUF

When searching for GGUF files in Hugging Face, we often find a GGUF file named like “Q4_K_M”.

- 4 indicates it applied 4-bit quantization for most parameters.
- ‘K’ indicates K-quant, which is a popular quantization method. K-quant applies “double quantization.”

K-quant (double quantization) in GGUF

When searching for GGUF files in Hugging Face, we often find a GGUF file named like “Q4_K_M”.

- 4 indicates it applied 4-bit quantization for most parameters.
- ‘K’ indicates K-quant, which is a popular quantization method. K-quant applies “double quantization.”
- ‘M’ indicates “medium,” indicating which parameters it allocates more bits in quantization. If it were ‘S’ (small), it means additional quantization bit allocation is applied on fewer parameters.

K-quant (double quantization) in GGUF

Definition (K-quant)

For a parameter vector θ , we partition the elements into 256-element super-blocks, and partition each super-block into 8 sub-blocks (each sub-block has 32 elements).

K-quant (double quantization) in GGUF

Definition (K-quant)

For a parameter vector θ , we partition the elements into 256-element super-blocks, and partition each super-block into 8 sub-blocks (each sub-block has 32 elements).

- The i -th super-block i stores two float16 numbers $d_i, d_{i,\min}$.

K-quant (double quantization) in GGUF

Definition (K-quant)

For a parameter vector θ , we partition the elements into 256-element super-blocks, and partition each super-block into 8 sub-blocks (each sub-block has 32 elements).

- The i -th super-block i stores two float16 numbers $d_i, d_{i,\min}$.
- Its j -th sub-block stores two 6-bit integers $m_{i,j}, s_{i,j} \in \{0, 1, \dots, 63\}$.

K-quant (double quantization) in GGUF

Definition (K-quant)

For a parameter vector θ , we partition the elements into 256-element super-blocks, and partition each super-block into 8 sub-blocks (each sub-block has 32 elements).

- The i -th super-block i stores two float16 numbers $d_i, d_{i,\min}$.
- Its j -th sub-block stores two 6-bit integers $m_{i,j}, s_{i,j} \in \{0, 1, \dots, 63\}$.
- Its r -th element stores 4-bit integer $q_{i,j,r} \in \{0, 1, \dots, 15\}$.

K-quant (double quantization) in GGUF

Definition (K-quant)

For a parameter vector θ , we partition the elements into 256-element super-blocks, and partition each super-block into 8 sub-blocks (each sub-block has 32 elements).

- The i -th super-block i stores two float16 numbers $d_i, d_{i,\min}$.
- Its j -th sub-block stores two 6-bit integers $m_{i,j}, s_{i,j} \in \{0, 1, \dots, 63\}$.
- Its r -th element stores 4-bit integer $q_{i,j,r} \in \{0, 1, \dots, 15\}$.

The dequantization mapping $\tilde{Q}_{i,j}$ is:

$$\tilde{Q}_{i,j}(q_{i,j,r}) = d_i s_{i,j} q_{i,j,r} + d_{i,\min} m_{i,j}$$

That is,

K-quant (double quantization) in GGUF

Definition (K-quant)

For a parameter vector θ , we partition the elements into 256-element super-blocks, and partition each super-block into 8 sub-blocks (each sub-block has 32 elements).

- The i -th super-block i stores two float16 numbers $d_i, d_{i,\min}$.
- Its j -th sub-block stores two 6-bit integers $m_{i,j}, s_{i,j} \in \{0, 1, \dots, 63\}$.
- Its r -th element stores 4-bit integer $q_{i,j,r} \in \{0, 1, \dots, 15\}$.

The dequantization mapping $\tilde{Q}_{i,j}$ is:

$$\tilde{Q}_{i,j}(q_{i,j,r}) = d_i s_{i,j} q_{i,j,r} + d_{i,\min} m_{i,j}$$

That is, the quantization width is given by $d_i s_{i,j}$ and the zero point is given by $d_{i,\min} m_{i,j}$.

Compression using knowledge distillation

Definition (Model Distillation)

Suppose we are given a function f_θ (the "teacher" model).

Compression using knowledge distillation

Definition (Model Distillation)

Suppose we are given a function f_{θ} (the "teacher" model).

For the purpose of approximating f_{θ} at a lower cost, we use another parametric function $g(\cdot)$ with a lower-dimensional parameter space (the "student" model).

Compression using knowledge distillation

Definition (Model Distillation)

Suppose we are given a function f_θ (the "teacher" model).

For the purpose of approximating f_θ at a lower cost, we use another parametric function $g(\cdot)$ with a lower-dimensional parameter space (the "student" model).

We then find a suitable parameter vector γ such that g_γ becomes a good approximation of f_θ . This process is called **model distillation**.

Compression using knowledge distillation

Definition (Model Distillation)

Suppose we are given a function f_θ (the "teacher" model).

For the purpose of approximating f_θ at a lower cost, we use another parametric function $g(\cdot)$ with a lower-dimensional parameter space (the "student" model).

We then find a suitable parameter vector γ such that g_γ becomes a good approximation of f_θ . This process is called **model distillation**.

Remark

This is a form of **model distillation**. Another form is **dataset distillation**, where the goal is to synthesize a small dataset that captures the learning effect of a large original dataset.

Compression - answer

Question: How can we reduce AI parameter memory/storage consumption, without compromising performance?

Answer:

Compression - answer

Question: How can we reduce AI parameter memory/storage consumption, without compromising performance?

Answer: We convert each real number parameter into a representation form using fewer bits, e.g.,

Compression - answer

Question: How can we reduce AI parameter memory/storage consumption, without compromising performance?

Answer: We convert each real number parameter into a representation form using fewer bits, e.g., a lower-precision floating point and fewer-bit quantization.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. How can we compress a NN? - Low precision and quantization

4. How can we evaluate the model modification? - Divergence

5. Summary

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How can we compress a NN? - Low precision and quantization
- 4. How can we evaluate the model modification? - Divergence**
5. Summary

Question: We often want to modify a NN model without changing its performance significantly. Compression is a typical case.

How

Question: We often want to modify a NN model without changing its performance significantly. Compression is a typical case.

How can we evaluate the model modification?

Divergence between probabilistic language models

The core of large language models is a probabilistic language model.

Divergence between probabilistic language models

The core of large language models is a probabilistic language model.

When this is compressed, one becomes concerned about how much it differs from the original.

Divergence between probabilistic language models

The core of large language models is a probabilistic language model.

When this is compressed, one becomes concerned about how much it differs from the original.

This chapter deals with metrics that express how much another probabilistic language model has diverged when there is a reference probabilistic language model.

A simple method: differences in individual evaluations

A simple method is to measure the performance of two models on the same task (e.g., multiple-choice questions) and compare the **difference**.

A simple method: differences in individual evaluations

A simple method is to measure the performance of two models on the same task (e.g., multiple-choice questions) and compare the **difference**.

However, if the evaluation is based solely on the answers, it is possible to overlook cases where the inference processes are significantly different even if the final answers are the same [1].

A simple method: differences in individual evaluations

A simple method is to measure the performance of two models on the same task (e.g., multiple-choice questions) and compare the **difference**.

However, if the evaluation is based solely on the answers, it is possible to overlook cases where the inference processes are significantly different even if the final answers are the same [1].

Therefore, one might consider directly comparing the probability distributions constituted by the probabilistic language models.

Review of probabilistic language models

Definition (Vocabulary and Token Sequence)

The set of possible values a token can take is called the **vocabulary**, denoted by $\mathcal{V}$ ¹. We identify $\mathcal{V}$ with $\{1, 2, \dots, D\}$.

¹In previous lectures, the set of nodes in a neural network was also denoted by $\mathcal{V}$, but since this lecture does not explicitly describe the graph structure of neural networks, $\mathcal{V}$ should always be taken to refer to the vocabulary.

Review of probabilistic language models

Definition (Vocabulary and Token Sequence)

The set of possible values a token can take is called the **vocabulary**, denoted by $\mathcal{V}^1$. We identify $\mathcal{V}$ with $\{1, 2, \dots, D\}$.

- The set of token sequences of length n is $\mathcal{V}^n$.
- The set of all token sequences of finite length is $\mathcal{V}^* = \mathcal{V}^0 \cup \mathcal{V}^1 \cup \mathcal{V}^2 \cup \dots$.
- For a sequence $t = (t_1, \dots, t_n)$, we use notations like $t_{<i}$ for the prefix $(t_1, \dots, t_{i-1})$.

¹In previous lectures, the set of nodes in a neural network was also denoted by $\mathcal{V}$, but since this lecture does not explicitly describe the graph structure of neural networks, $\mathcal{V}$ should always be taken to refer to the vocabulary.

Review of probabilistic language models

Definition (Probabilistic Language Model (most general form))

A **probabilistic language model** is a function $P(\cdot|\cdot)$ that, given any finite-length token sequence $t \in \mathcal{V}^*$, returns the probability mass function of the next token, conditioned on it.

Review of probabilistic language models

Definition (Probabilistic Language Model (most general form))

A **probabilistic language model** is a function $P(\cdot|\cdot)$ that, given any finite-length token sequence $\mathbf{t} \in \mathcal{V}^*$, returns the probability mass function of the next token, conditioned on it.

More formally, for any $\mathbf{t} \in \mathcal{V}^*$, the following must hold:

$$\sum_{v \in \mathcal{V}} P(v \mid \mathbf{t}) = 1 \quad (4)$$

Review of probabilistic language models

Definition (Probabilistic Language Model (most general form))

A **probabilistic language model** is a function $P(\cdot|\cdot)$ that, given any finite-length token sequence $\mathbf{t} \in \mathcal{V}^*$, returns the probability mass function of the next token, conditioned on it.

More formally, for any $\mathbf{t} \in \mathcal{V}^*$, the following must hold:

$$\sum_{v \in \mathcal{V}} P(v | \mathbf{t}) = 1 \quad (4)$$

Since a probabilistic language model is a conditional probability mass function, the problem reduces to quantifying the divergence between general probability mass functions.

How to quantify the divergence of probability mass functions?

Given a reference probability mass function P and another one Q , we want to measure how much Q diverges from P .

How to quantify the divergence of probability mass functions?

Given a reference probability mass function P and another one Q , we want to measure how much Q diverges from P .

If Q is close to P , then for an outcome z where $P(z)$ is large, $Q(z)$ should also be large.

How to quantify the divergence of probability mass functions?

Given a reference probability mass function P and another one Q , we want to measure how much Q diverges from P .

If Q is close to P , then for an outcome z where $P(z)$ is large, $Q(z)$ should also be large.

This suggests a divergence criterion of the form:

$$\mathbb{E}_{Z \sim P} [\phi(Q(Z))] - C \tag{5}$$

using a monotonically decreasing function ϕ to penalize small probability masses.

How to quantify the divergence of probability mass functions?

Definition (Axiomatization of Divergence Functional)

For $\Delta_\phi(P \parallel Q) := \mathbb{E}_{Z \sim P}[\phi(Q(Z))] - C$, we impose:

(A1) **Reflexivity:** $\Delta_\phi(P \parallel P) = 0$.

(A2) **Non-negativity:** $\Delta_\phi(P \parallel Q) \geq 0$, with equality iff $P = Q$.

(A3) **Continuity:** $\Delta_\phi(P \parallel Q)$ is continuous in Q .

(A4) **Additivity over independent products:** For product distributions $P_1 \otimes P_2$ and $Q_1 \otimes Q_2$,

$$\Delta_\phi(P_1 \otimes P_2 \parallel Q_1 \otimes Q_2) = \Delta_\phi(P_1 \parallel Q_1) + \Delta_\phi(P_2 \parallel Q_2)$$

How to quantify the divergence of probability mass functions?

Theorem (Uniqueness of KL Divergence)

For a monotonically decreasing continuous function ϕ satisfying axioms (A1) – (A4), there exist constants $c > 0$ and B such that

$$\phi(u) = -c \log u + B \quad (u \in (0, 1]) \quad (6)$$

How to quantify the divergence of probability mass functions?

Theorem (Uniqueness of KL Divergence)

For a monotonically decreasing continuous function ϕ satisfying axioms (A1) – (A4), there exist constants $c > 0$ and B such that

$$\phi(u) = -c \log u + B \quad (u \in (0, 1]) \quad (6)$$

This implies that the divergence must be proportional to the KL divergence:

$$\Delta_{\phi}(P \parallel Q) = c \mathbb{E}_P \left[\log \frac{P(Z)}{Q(Z)} \right] = c D_{\text{KL}}(P \parallel Q) \quad (7)$$

How to quantify the divergence of probability mass functions?

The proof is insightful:

- **Step 1:** The **additivity** axiom (A4) forces ϕ to satisfy a functional equation:
$$\phi(uv) - \phi(u) - \phi(v) = \text{constant}.$$

How to quantify the divergence of probability mass functions?

The proof is insightful:

- **Step 1:** The **additivity** axiom (A4) forces ϕ to satisfy a functional equation:
$$\phi(uv) - \phi(u) - \phi(v) = \text{constant}.$$
- **Step 2:** This functional equation, combined with the **continuity** axiom (A3), implies that ϕ must be a logarithmic function. This is a classic result related to Cauchy's functional equation.

How to quantify the divergence of probability mass functions?

The proof is insightful:

- **Step 1:** The **additivity** axiom (A4) forces ϕ to satisfy a functional equation:
$$\phi(uv) - \phi(u) - \phi(v) = \text{constant}.$$
- **Step 2:** This functional equation, combined with the **continuity** axiom (A3), implies that ϕ must be a logarithmic function. This is a classic result related to Cauchy's functional equation.
- **Step 3:** The **reflexivity** (A1) and **non-negativity** (A2) axioms fix the constants and ensure the result is proportional to the standard KL divergence, with a positive coefficient.

Definition of Kullback-Leibler (KL) divergence

Definition (KL Divergence (for pmfs))

Let P, Q be two probability mass functions on a finite set $\mathcal{S}$. The **KL divergence (relative entropy)** is defined as

$$D_{\text{KL}}(P \parallel Q) := \sum_{z \in \mathcal{S}} P(z) \log \left(\frac{P(z)}{Q(z)} \right) \in [0, \infty] \quad (8)$$

It is always non-negative, and $D_{\text{KL}}(P \parallel Q) = 0$ if and only if $P = Q$ [7, 3].

Definition of Kullback-Leibler (KL) divergence

Example (Complete Numerical Example of KL)

Consider the probability distributions on $\{a, b, c\}$:

$$P = (0.5, 0.3, 0.2)$$

$$Q = (0.4, 0.4, 0.2)$$

Definition of Kullback-Leibler (KL) divergence

Example (Complete Numerical Example of KL)

Consider the probability distributions on $\{a, b, c\}$:

$$P = (0.5, 0.3, 0.2)$$

$$Q = (0.4, 0.4, 0.2)$$

From the definition:

$$\begin{aligned} D_{\text{KL}}(P \parallel Q) &= \sum_{x \in \{a, b, c\}} P(x) \log \frac{P(x)}{Q(x)} \\ &= 0.5 \log \frac{0.5}{0.4} + 0.3 \log \frac{0.3}{0.4} + 0.2 \log \frac{0.2}{0.2} \end{aligned} \quad (9)$$

Definition of Kullback-Leibler (KL) divergence

The last term is $0.2 \log(1) = 0$.

Definition of Kullback-Leibler (KL) divergence

The last term is $0.2 \log(1) = 0$.

Using the natural logarithm:

$$0.5 \log(1.25) \approx 0.5 \times 0.22314 \approx 0.11157$$

$$0.3 \log(0.75) \approx 0.3 \times (-0.28768) \approx -0.08630$$

Definition of Kullback-Leibler (KL) divergence

The last term is $0.2 \log(1) = 0$.

Using the natural logarithm:

$$0.5 \log(1.25) \approx 0.5 \times 0.22314 \approx 0.11157$$

$$0.3 \log(0.75) \approx 0.3 \times (-0.28768) \approx -0.08630$$

Summing them up:

$$D_{\text{KL}}(P \parallel Q) \approx 0.11157 - 0.08630 = 0.02527 \text{ [nats]} \quad (10)$$

Definition of Kullback-Leibler (KL) divergence

Exercise (Numerical Calculation of KL)

On the vocabulary $\{x_1, x_2, x_3\}$, let

$$P = (0.2, 0.5, 0.3), \quad Q = (0.1, 0.7, 0.2)$$

Calculate $D_{\text{KL}}(P \parallel Q)$ using the natural logarithm.

Definition of Kullback-Leibler (KL) divergence

Answer

$$\begin{aligned}D_{\text{KL}}(P \parallel Q) &= 0.2 \log \frac{0.2}{0.1} + 0.5 \log \frac{0.5}{0.7} + 0.3 \log \frac{0.3}{0.2} \\&= 0.2 \log 2 + 0.5 \log(5/7) + 0.3 \log(3/2)\end{aligned}$$

Numerically, this is:

$$\begin{aligned}&\approx 0.2 \times (0.69315) + 0.5 \times (-0.33647) + 0.3 \times (0.40547) \\&\approx 0.13863 - 0.16824 + 0.12164 \approx 0.09203 \text{ [nats]}\end{aligned}$$

Extension of KL to language models (conditional distributions)

How do we apply KL divergence to probabilistic language models P and Q , which are conditional distributions?

Extension of KL to language models (conditional distributions)

How do we apply KL divergence to probabilistic language models P and Q , which are conditional distributions?

There are two main, closely related ways.

Extension of KL to language models (conditional distributions)

How do we apply KL divergence to probabilistic language models P and Q , which are conditional distributions?

There are two main, closely related ways.

Extension of KL to language models (conditional distributions)

How do we apply KL divergence to probabilistic language models P and Q , which are conditional distributions?

There are two main, closely related ways.

- “Direct” KL divergence.

Extension of KL to language models (conditional distributions)

How do we apply KL divergence to probabilistic language models P and Q , which are conditional distributions?

There are two main, closely related ways.

- “Direct” KL divergence.
- KL divergence between the next token distributions on a dataset. (Practically more common)

Extension of KL to language models (conditional distributions)

Definition (KL divergence based on Joint Distribution)

For a fixed length n , we can define the **joint distributions** over sequences of length n induced by the language models P and Q .

$$P^{(n)}(\mathbf{t}_{1:n}) := \prod_{i=1}^n P(t_i \mid \mathbf{t}_{<i}) \quad (11)$$

(and similarly for $Q^{(n)}$).

Extension of KL to language models (conditional distributions)

Definition (KL divergence based on Joint Distribution)

For a fixed length n , we can define the **joint distributions** over sequences of length n induced by the language models P and Q .

$$P^{(n)}(\mathbf{t}_{1:n}) := \prod_{i=1}^n P(t_i \mid \mathbf{t}_{<i}) \quad (11)$$

(and similarly for $Q^{(n)}$).

Then we define the KL divergence between these two joint distributions:

$$D_{\text{KL}}(P^{(n)} \parallel Q^{(n)}) = \sum_{\mathbf{t}_{1:n}} P^{(n)}(\mathbf{t}_{1:n}) \log \frac{P^{(n)}(\mathbf{t}_{1:n})}{Q^{(n)}(\mathbf{t}_{1:n})} \quad (12)$$

Extension of KL to language models (conditional distributions)

This joint KL can be decomposed using a chain rule.

Extension of KL to language models (conditional distributions)

This joint KL can be decomposed using a chain rule.

Proposition (Chain Rule for KL Divergence)

For any $n \in \mathbb{Z}_{>0}$,

$$D_{\text{KL}}(P^{(n)} \parallel Q^{(n)}) = \sum_{i=1}^n \mathbb{E}_{\mathbf{t}_{<i} \sim P^{(i-1)}} [D_{\text{KL}}(P(\cdot \mid \mathbf{t}_{<i}) \parallel Q(\cdot \mid \mathbf{t}_{<i}))] \quad (13)$$

This is the sum of expected conditional KL divergences at each step.

Extension of KL to language models (conditional distributions)

Definition (KL divergence between next-token distributions on a dataset)

A more practical approach is to compute the average KL divergence over a validation dataset $\mathcal{D} = \{\mathbf{t}^{(j)}\}_{j=1}^N$.

Extension of KL to language models (conditional distributions)

Definition (KL divergence between next-token distributions on a dataset)

A more practical approach is to compute the average KL divergence over a validation dataset $\mathcal{D} = \{\mathbf{t}^{(j)}\}_{j=1}^N$.

We average the KL divergence of the next-token predictions over all positions in all sequences in the dataset:

$$\hat{D}_{\text{KL}}^{\mathcal{D}}(P \parallel Q) := \frac{1}{|\mathcal{D}|_{\text{tokens}}} \sum_{j=1}^N \sum_{i=1}^{|\mathbf{y}^{(j)}|} D_{\text{KL}}\left(P(\cdot \mid \text{context}) \parallel Q(\cdot \mid \text{context})\right). \quad (14)$$

Extension of KL to language models (conditional distributions)

These two definitions are closely related.

Extension of KL to language models (conditional distributions)

These two definitions are closely related.

Proposition (Virtual Equivalence of A and B)

If the dataset $\mathcal{D}$ is generated i.i.d. according to the model P , then the expected value of the dataset-based KL divergence (B) is equal to the per-token joint KL divergence (A).

$$\mathbb{E}_{\mathcal{D} \sim (P^{(n)})^{\otimes N}} \left[\hat{D}_{\text{KL}}^{\mathcal{D}}(P \parallel Q) \right] = \frac{1}{n} D_{\text{KL}}(P^{(n)} \parallel Q^{(n)}). \quad (15)$$

Extension of KL to language models (conditional distributions)

Example (Numerical Example of Sequential KL (length 2))

Let $\mathcal{V} = \{A, B\}$, $n = 2$. The conditional distributions are:

$$P(A \mid ()) = 0.6, P(A \mid A) = 0.7, P(A \mid B) = 0.2$$

$$Q(A \mid ()) = 0.5, Q(A \mid A) = 0.6, Q(A \mid B) = 0.3$$

(The probabilities for B are just 1 minus these values). Let's calculate $D_{\text{KL}}(P^{(2)} \parallel Q^{(2)})$.

Extension of KL to language models (conditional distributions)

Step 1: KL for the first token ($i = 1$)

The context is the empty sequence $()$.

$$\begin{aligned} D_{\text{KL}}(P(\cdot | ()) \parallel Q(\cdot | ())) &= 0.6 \log \frac{0.6}{0.5} + 0.4 \log \frac{0.4}{0.5} \\ &\approx 0.0204 \end{aligned}$$

Extension of KL to language models (conditional distributions)

Step 2: Expected KL for the second token ($i = 2$)

We need to average the conditional KL over the first token, drawn from $P(\cdot \mid ())$.

$$\begin{aligned}\mathbb{E}_{t_1 \sim P} [D_{\text{KL}}(P(\cdot \mid t_1) \parallel Q(\cdot \mid t_1))] \\&= P(A \mid ()) \cdot D_{\text{KL}}(P(\cdot \mid A) \parallel Q(\cdot \mid A)) \\&\quad + P(B \mid ()) \cdot D_{\text{KL}}(P(\cdot \mid B) \parallel Q(\cdot \mid B))\end{aligned}$$

Extension of KL to language models (conditional distributions)

Step 2: Expected KL for the second token ($i = 2$)

We need to average the conditional KL over the first token, drawn from $P(\cdot \mid ())$.

$$\begin{aligned}\mathbb{E}_{t_1 \sim P} [D_{\text{KL}}(P(\cdot \mid t_1) \parallel Q(\cdot \mid t_1))] \\&= P(A \mid ()) \cdot D_{\text{KL}}(P(\cdot \mid A) \parallel Q(\cdot \mid A)) \\&\quad + P(B \mid ()) \cdot D_{\text{KL}}(P(\cdot \mid B) \parallel Q(\cdot \mid B))\end{aligned}$$

$$\begin{aligned}&= 0.6 \cdot \left(0.7 \log \frac{0.7}{0.6} + 0.3 \log \frac{0.3}{0.4} \right) \\&\quad + 0.4 \cdot \left(0.2 \log \frac{0.2}{0.3} + 0.8 \log \frac{0.8}{0.7} \right) \approx 0.0125\end{aligned}$$

Extension of KL to language models (conditional distributions)

Step 3: Total KL Divergence

Summing the results from both steps:

$$\begin{aligned}D_{\text{KL}}(P^{(2)} \parallel Q^{(2)}) &= (\text{KL at } i = 1) + (\text{Expected KL at } i = 2) \\&\approx 0.0204 + 0.0125 \\&= 0.0329\end{aligned}$$

Extension of KL to language models (conditional distributions)

Exercise (Sequential KL Calculation Practice)

Let $\mathcal{V} = \{0, 1\}$, $n = 2$.

$$P(1 \mid ()) = 0.3, \quad P(1 \mid 1) = 0.6, \quad P(1 \mid 0) = 0.2$$

$$Q(1 \mid ()) = 0.4, \quad Q(1 \mid 1) = 0.5, \quad Q(1 \mid 0) = 0.3$$

Find $D_{\text{KL}}(P^{(2)} \parallel Q^{(2)})$.

Extension of KL to language models (conditional distributions)

Answer

First token ($i = 1$):

$$D_{\text{KL}}(P(\cdot|()) \parallel Q(\cdot|())) = 0.3 \log \frac{0.3}{0.4} + 0.7 \log \frac{0.7}{0.6} \approx 0.0224$$

Extension of KL to language models (conditional distributions)

Answer

First token ($i = 1$):

$$D_{\text{KL}}(P(\cdot|()) \parallel Q(\cdot|())) = 0.3 \log \frac{0.3}{0.4} + 0.7 \log \frac{0.7}{0.6} \approx 0.0224$$

Expected KL for second token ($i = 2$):

$$\begin{aligned}\mathbb{E}_{t_1 \sim P} &= 0.3 \cdot \left[0.6 \log \frac{0.6}{0.5} + 0.4 \log \frac{0.4}{0.5} \right] \\ &\quad + 0.7 \cdot \left[0.2 \log \frac{0.2}{0.3} + 0.8 \log \frac{0.8}{0.7} \right] \\ &\approx 0.3 \cdot (0.0201) + 0.7 \cdot (-0.0033) \approx 0.0037\end{aligned}$$

Extension of KL to language models (conditional distributions)

Answer

First token ($i = 1$):

$$D_{\text{KL}}(P(\cdot|()) \parallel Q(\cdot|())) = 0.3 \log \frac{0.3}{0.4} + 0.7 \log \frac{0.7}{0.6} \approx 0.0224$$

Expected KL for second token ($i = 2$):

$$\begin{aligned} \mathbb{E}_{t_1 \sim P} &= 0.3 \cdot \left[0.6 \log \frac{0.6}{0.5} + 0.4 \log \frac{0.4}{0.5} \right] \\ &\quad + 0.7 \cdot \left[0.2 \log \frac{0.2}{0.3} + 0.8 \log \frac{0.8}{0.7} \right] \\ &\approx 0.3 \cdot (0.0201) + 0.7 \cdot (-0.0033) \approx 0.0037 \end{aligned}$$

Total KL:

$$D_{\text{KL}}(P^{(2)} \parallel Q^{(2)}) \approx 0.0224 + 0.0037 = 0.0261 \text{ [nats]}$$

Definition of Jensen-Shannon (JS) divergence

KL divergence is not symmetric. A symmetric version is the JS divergence.

Definition of Jensen-Shannon (JS) divergence

KL divergence is not symmetric. A symmetric version is the JS divergence.

Definition (JS Divergence)

For the **mixture distribution** $M := \frac{1}{2}(P + Q)$ of P, Q ,

$$D_{\text{JS}}(P \parallel Q) := \frac{1}{2}D_{\text{KL}}(P \parallel M) + \frac{1}{2}D_{\text{KL}}(Q \parallel M) \quad (16)$$

is called the **Jensen-Shannon divergence**. D_{JS} is symmetric and bounded $[0, \log 2]$ [8].

Definition of Jensen-Shannon (JS) divergence

KL divergence is not symmetric. A symmetric version is the JS divergence.

Definition (JS Divergence)

For the **mixture distribution** $M := \frac{1}{2}(P + Q)$ of P, Q ,

$$D_{\text{JS}}(P \parallel Q) := \frac{1}{2}D_{\text{KL}}(P \parallel M) + \frac{1}{2}D_{\text{KL}}(Q \parallel M) \quad (16)$$

is called the **Jensen-Shannon divergence**. D_{JS} is symmetric and bounded $[0, \log 2]$ [8].

Remark

The square root of the JS divergence, $\sqrt{D_{\text{JS}}}$, satisfies the axioms of a **metric**, including the triangle inequality [8].

Definition of Jensen-Shannon (JS) divergence

Example (Complete Numerical Example of JS)

Using P, Q from the KL example:

$$P = (0.5, 0.3, 0.2)$$

$$Q = (0.4, 0.4, 0.2)$$

First, find the mixture distribution $M = \frac{1}{2}(P + Q)$.

Definition of Jensen-Shannon (JS) divergence

Example (Complete Numerical Example of JS)

Using P, Q from the KL example:

$$P = (0.5, 0.3, 0.2)$$

$$Q = (0.4, 0.4, 0.2)$$

First, find the mixture distribution $M = \frac{1}{2}(P + Q)$.

$$M = (0.45, 0.35, 0.2)$$

Definition of Jensen-Shannon (JS) divergence

Example (Complete Numerical Example of JS)

Using P, Q from the KL example:

$$P = (0.5, 0.3, 0.2)$$

$$Q = (0.4, 0.4, 0.2)$$

First, find the mixture distribution $M = \frac{1}{2}(P + Q)$.

$$M = (0.45, 0.35, 0.2)$$

Now, calculate $D_{\text{JS}}(P \parallel Q) = \frac{1}{2}D_{\text{KL}}(P \parallel M) + \frac{1}{2}D_{\text{KL}}(Q \parallel M)$.

Definition of Jensen-Shannon (JS) divergence

Step 1: Calculate $D_{\text{KL}}(P \parallel M)$:

$$\begin{aligned} D_{\text{KL}}(P \parallel M) &= 0.5 \log \frac{0.5}{0.45} + 0.3 \log \frac{0.3}{0.35} + 0.2 \log \frac{0.2}{0.2} \\ &\approx 0.0527 - 0.0463 + 0 = 0.0064 \end{aligned}$$

Definition of Jensen-Shannon (JS) divergence

Step 1: Calculate $D_{\text{KL}}(P \parallel M)$:

$$\begin{aligned} D_{\text{KL}}(P \parallel M) &= 0.5 \log \frac{0.5}{0.45} + 0.3 \log \frac{0.3}{0.35} + 0.2 \log \frac{0.2}{0.2} \\ &\approx 0.0527 - 0.0463 + 0 = 0.0064 \end{aligned}$$

Step 2: Calculate $D_{\text{KL}}(Q \parallel M)$:

$$\begin{aligned} D_{\text{KL}}(Q \parallel M) &= 0.4 \log \frac{0.4}{0.45} + 0.4 \log \frac{0.4}{0.35} + 0.2 \log \frac{0.2}{0.2} \\ &\approx -0.0472 + 0.0536 + 0 = 0.0064 \end{aligned}$$

Definition of Jensen-Shannon (JS) divergence

Step 1: Calculate $D_{\text{KL}}(P \parallel M)$:

$$\begin{aligned} D_{\text{KL}}(P \parallel M) &= 0.5 \log \frac{0.5}{0.45} + 0.3 \log \frac{0.3}{0.35} + 0.2 \log \frac{0.2}{0.2} \\ &\approx 0.0527 - 0.0463 + 0 = 0.0064 \end{aligned}$$

Step 2: Calculate $D_{\text{KL}}(Q \parallel M)$:

$$\begin{aligned} D_{\text{KL}}(Q \parallel M) &= 0.4 \log \frac{0.4}{0.45} + 0.4 \log \frac{0.4}{0.35} + 0.2 \log \frac{0.2}{0.2} \\ &\approx -0.0472 + 0.0536 + 0 = 0.0064 \end{aligned}$$

Step 3: Average them:

$$D_{\text{JS}}(P \parallel Q) = \frac{1}{2}(0.0064 + 0.0064) = 0.0064 \text{ [nats]}$$

Definition of Jensen-Shannon (JS) divergence

Exercise (Numerical Calculation of JS)

For $P = (0.2, 0.5, 0.3)$ and $Q = (0.1, 0.7, 0.2)$ from the KL exercise, find $D_{\text{JS}}(P \parallel Q)$.

Definition of Jensen-Shannon (JS) divergence

Answer

Mixture: $M = \frac{1}{2}(P + Q) = (0.15, 0.6, 0.25)$.

Definition of Jensen-Shannon (JS) divergence

Answer

Mixture: $M = \frac{1}{2}(P + Q) = (0.15, 0.6, 0.25)$.

KLs:

$$\begin{aligned}D_{\text{KL}}(P \parallel M) &= 0.2 \log \frac{0.2}{0.15} + 0.5 \log \frac{0.5}{0.6} + 0.3 \log \frac{0.3}{0.25} \\&\approx 0.0575 - 0.0912 + 0.0547 = 0.021\end{aligned}$$

$$\begin{aligned}D_{\text{KL}}(Q \parallel M) &= 0.1 \log \frac{0.1}{0.15} + 0.7 \log \frac{0.7}{0.6} + 0.2 \log \frac{0.2}{0.25} \\&\approx -0.0405 + 0.1079 - 0.0446 = 0.0228\end{aligned}$$

Definition of Jensen-Shannon (JS) divergence

Answer

Mixture: $M = \frac{1}{2}(P + Q) = (0.15, 0.6, 0.25)$.

KLs:

$$\begin{aligned}D_{\text{KL}}(P \parallel M) &= 0.2 \log \frac{0.2}{0.15} + 0.5 \log \frac{0.5}{0.6} + 0.3 \log \frac{0.3}{0.25} \\&\approx 0.0575 - 0.0912 + 0.0547 = 0.021\end{aligned}$$

$$\begin{aligned}D_{\text{KL}}(Q \parallel M) &= 0.1 \log \frac{0.1}{0.15} + 0.7 \log \frac{0.7}{0.6} + 0.2 \log \frac{0.2}{0.25} \\&\approx -0.0405 + 0.1079 - 0.0446 = 0.0228\end{aligned}$$

JS Divergence:

$$D_{\text{JS}}(P \parallel Q) = \frac{1}{2}(0.021 + 0.0228) \approx 0.0219 \text{ [nats]}$$

Definition of Jensen-Shannon (JS) divergence

Similar to KL, JS divergence can be extended to language models by considering either the **joint distribution** over sequences or by averaging over a **dataset**.

Definition of Jensen-Shannon (JS) divergence

Similar to KL, JS divergence can be extended to language models by considering either the **joint distribution** over sequences or by averaging over a **dataset**.

A similar **chain rule** also holds, decomposing the total JS divergence into a sum of expected conditional JS divergences at each position.

$$D_{\text{JS}}(P^{(n)} \parallel Q^{(n)}) = \sum_{i=1}^n \left\{ \frac{1}{2} \mathbb{E}_{P^{(i-1)}}[D_{\text{KL}}(P|\text{prefix}) \parallel M|\text{prefix})] \right. \\ \left. + \frac{1}{2} \mathbb{E}_{Q^{(i-1)}}[D_{\text{KL}}(Q|\text{prefix}) \parallel M|\text{prefix})] \right\}$$

Evaluating the model difference - answer

Question: How can we evaluate the model modification?

Answer:

Evaluating the model difference - answer

Question: How can we evaluate the model modification?

Answer: We can calculate the **divergence** between the probabilistic distribution constructed by the models (before/after the modification).

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How can we compress a NN? - Low precision and quantization
- 4. How can we evaluate the model modification? - Divergence**
5. Summary

Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How can we compress a NN? - Low precision and quantization
4. How can we evaluate the model modification? - Divergence
- 5. Summary**

Summary

Let's summarize the key takeaways from today's lecture.

- **Model compression** is an attempt to reduce the model scale while preserving its input-output relationship. Examples include low-precision floating number methods, quantization, and distillation.

Summary

Let's summarize the key takeaways from today's lecture.

- **Model compression** is an attempt to reduce the model scale while preserving its input-output relationship. Examples include low-precision floating number methods, quantization, and distillation.
- The **difference between probabilistic language models** can be measured using information-theoretic divergences.

Summary

Let's summarize the key takeaways from today's lecture.

- **Model compression** is an attempt to reduce the model scale while preserving its input-output relationship. Examples include low-precision floating number methods, quantization, and distillation.
- The **difference between probabilistic language models** can be measured using information-theoretic divergences.
- The **KL divergence** is the expectation-type divergence satisfying reflectivity, non-negativity, continuity, and additivity. Its symmetrized version is known as the **JS divergence**.

References i

- [1] Rishiraj Acharya.

Why maybe we're measuring llm compression wrong.

<https://huggingface.co/blog/rishiraj/kld-guided-quantization>, 2025.
Blog article.

- [2] Cristian Buciluă, Rich Caruana, and Alexandru Niculescu-Mizil.

Model compression.

In Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), pages 535–541, 2006.

- [3] Thomas M. Cover and Joy A. Thomas.

Elements of Information Theory.

Wiley-Interscience, 2 edition, 2006.

References ii

- [4] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean.
Distilling the knowledge in a neural network.
In NIPS Deep Learning and Representation Learning Workshop, 2015.
arXiv:1503.02531.
- [5] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio.
Quantized neural networks: Training neural networks with low precision weights and activations.
Journal of Machine Learning Research, 18(187):1–30, 2018.
Earlier version: arXiv:1609.07061 (2016).

References iii

- [6] Benoit Jacob, Skirmantas Kligys, Bo Chen, et al.
Quantization and training of neural networks for efficient integer-arithmetic-only inference.
In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pages 2704–2713, 2018.
- [7] Solomon Kullback and Richard A. Leibler.
On information and sufficiency.
Annals of Mathematical Statistics, 22(1):79–86, 1951.
- [8] Jianhua Lin.
Divergence measures based on the shannon entropy.
IEEE Transactions on Information Theory, 37(1):145–151, 1991.

References iv

- [9] Paulius Micikevicius, Sharan Narang, Jonah Alben, et al.
Mixed precision training.
In International Conference on Learning Representations (ICLR) Workshop,
2018.
arXiv:1710.03740.